

TD5 : Recherche du plus court chemin dans un graphe orienté

— Rappel : le cas des valeurs positives sur les arcs

Algorithm 1 Dijkstra_a (G, l, s).

Require: Graph $G = (V, E)$ with positive edge weights $\{l_e > 0 : e \in E\}$, vertex $s \in V$

Ensure: $\forall u \in V$, $dist(u)$ is set to the shortest distance from s to u ; $prev(u)$ points to the previous of u on this shortest path

```

1: Initialisation :  $\forall u \in V : dist(u) \leftarrow \infty$  and  $prev(u) \leftarrow nil$ ;  $dist(s) \leftarrow 0$ ;
2:  $P \leftarrow \emptyset$ , ( $P$  contient les sommets  $v$ , tq  $dist(v)$  est calculée définitivement)
3:  $T \leftarrow V$  ( $T$  contient des sommets  $v$ , tq  $dist(v)$  n'est qu'une borne supérieure)
4: while  $T$  is not empty do
5:    $u \leftarrow \underset{v \in T}{\operatorname{argmin}} dist(v)$ 
6:    $P \leftarrow P \cup \{u\}$ 
7:    $T \leftarrow T \setminus \{u\}$ 
8:   for all edges  $(u, v) \in E$  do
9:     if  $dist(v) > dist(u) + l(u, v)$  then
10:        $dist(v) = dist(u) + l(u, v)$ 
11:        $prev(v) \leftarrow u$ 
12:     end if
13:   end for
14: end while
```

Exercice 1 (Algorithme de Dijkstra).

1. Appliquer l'algorithme de Dijkstra au graphe de la figure 1 afin de calculer les chemins de poids minimal entre A et les autres sommets du graphe. À la fin du calcul :
 - chaque sommet i aura le couple $(dist(i), pred(i))$;
 - l'arborescence qui donne les chemins les plus courts de A vers tous les autres sommets sera mise en évidence (en gras par exemple).

Solution 1. D  rouler l'algorithme de Dijkstra, en veillant bien    indiquer P (« permanents » : sommets trait  s) et les pr  d  cesseurs :

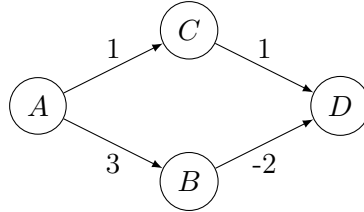
it��ration	P	T					
		A	B	C	D	E	F
0	$\emptyset$	0	∞	∞	∞	∞	∞
1	$\{A\}$	—	250_A	1000 _A	∞	∞	∞
2	$\{A; B\}$	—	—	950 _B	400_B	∞	∞
3	$\{A; B; D\}$	—	—	950_B	—	∞	1401 _D
4	$\{A; B; D; C\}$	—	—	—	—	∞	1277_C
5	$\{A; B; D; C; F\}$	—	—	—	—	1611_F	—

2. Quelle est la relation entre $dist(x)$, $dist(y)$ et $l(x, y)$ pour tout arc (x, y) de l'arborescence fabriqu  e par l'algorithme de Dijkstra ? Notons cette relation R .

Solution 2. $xRy : dist(x) + l(x, y) = dist(y)$ (remarque : hors de cette arborescence, on a tout de m  me $dist(x) + l(x, y) \geq dist(y)$)

3. Montrer    l'aide d'un exemple que lorsque les poids peuvent   tre n  gatifs, l'algorithme de Dijkstra ne donne pas n  cessairement les plus courts chemins. Est-ce que la relation R est vraie dans ce cas aussi ?

Solution 3. Contre-exemple :



avec quatre sommets et quatre arcs :

$$V = \{A; B; C; D\}, E = \{(A, B)_3; (A, C)_1; (B, D)_{-2}; (C, D)_1\}$$

l'algorithme de Dijkstra donne ici pour aller de A    D le chemin $A \rightarrow C \rightarrow D$, qui n'est pas un plus court chemin (contrairement    $A \rightarrow B \rightarrow D$).

R ne tient plus n  cessairement, puisque l'on a ici $dist(B) + l(B, D) = 3 + (-2) = 1 < 2 = dist(D)$.

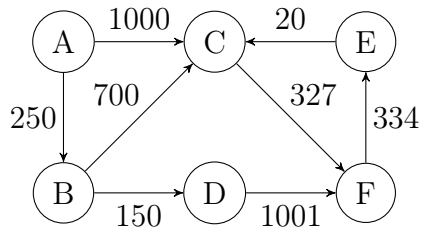


FIGURE 1 – Graphe pour l'exercice 1.

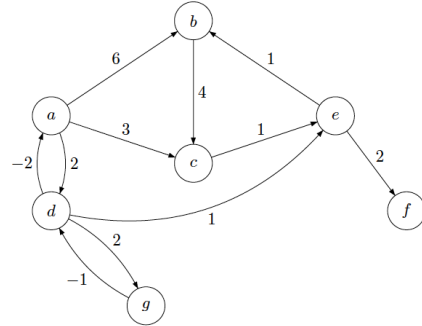


FIGURE 2 – Graphe avec des arcs pondérés dans $\mathbb{Z}$

Exercice 2 (Chemin de valeur optimale dans un graphe avec des arcs de longueur/poids quelconque). On considère le graphe de la figure 2

- 1) Il s'agit d'obtenir les chemins de valeur minimale issus de a et joignant tous les sommets. Quels sont les algorithmes vus en cours pour résoudre ce problème? Connaissez-vous plusieurs variantes de ces algorithmes? Si oui, appliquer la variante la plus adaptée au cas considéré. Justifier.

Solution 1. On a vu en cours les algorithmes de Dijkstra et de Bellman-Ford pour calculer des plus courts chemins dans des graphes. Ici le plus adapté est Bellman-Ford car Dijkstra a besoin d'avoir un graphe sans poids strictement négatif en entrée. Voici le déroulement de l'algorithme de Bellman-Ford, avec un arrêt à partir de $k = 4$ car on a stabilisé :

k	stable	sommets						
		a	b	c	d	e	f	g
0	false	0	∞	∞	∞	∞	∞	∞
1	false	—	6	3	2	—	—	—
2	false	—	—	—	—	3	—	4
3	false	—	4	—	—	—	5	—
4	true	—	—	—	—	—	—	—

- 2) Comment modifier cet algorithme de façon à ce qu'il s'applique au problème de la maximisation? À quelle condition cet algorithme est-il applicable? Appliquer l'algorithme modifié sur le graphe ci-dessus.

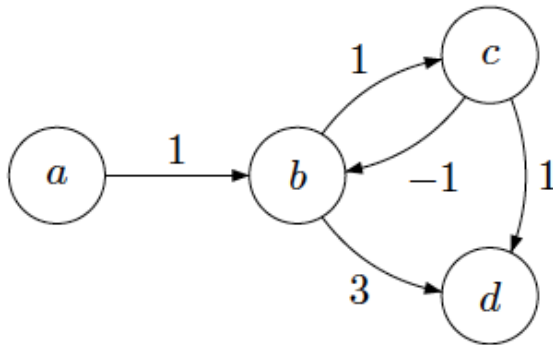
Solution 2. Pour adapter l'algorithme de Bellman-Ford pour le cas de la maximisation, il suffit d'initialiser $dist := -\infty$ pour chaque sommet (sauf pour le sommet initial) et d'utiliser l'opérateur max à la place de min. La condition d'applicabilité est l'absence d'un circuit de poids strictement positif.

Pour y voir plus clair pour cette condition, on peut dire que cela revient à inverser le graphe en mettant l'opposé de tous les poids (1 devient -1 par exemple). En effet un chemin maximal dans le graphe opposé correspond à un chemin minimal dans le graphe normal. Cela revient bien à vérifier qu'il n'y a pas de circuit de poids strictement positif.

- 3) En ajoutant une constante suffisamment grande au poids de chaque arc, on peut rendre ce poids positif. On décide alors d'appliquer l'algorithme de Dijkstra au nouveau graphe pondéré pour trouver les plus courts chemins de a à tout sommet. Cette idée est-elle bonne? Justifiez.

Solution 3. C'est une très mauvaise idée! En rajoutant une constante au poids de chaque arc, on ne transforme pas le problème initial en un problème équivalent, car l'augmentation du poids d'un chemin dépend de son nombre d'arcs, ce qui peut modifier les plus courts chemins. Or deux problèmes sont équivalents si, et seulement si toute solution de l'un donne une solution de l'autre. L'équivalence n'étant pas respectée, obtenir une solution de l'un ne donnera pas la solution pour l'autre. Cette idée est donc mauvaise au départ indépendamment des algorithmes.

On peut par exemple donner un cas qui ne marche pas en considérant le graphe suivant, pour lequel le plus court chemin menant de a à d n'est pas le bon si on applique l'algorithme de Dijkstra après avoir ajouté 2 au poids de tous les arcs (et on trouve deux plus courts chemins potentiels si on ajoute 1, ce qui n'est pas dans le graphe initial) :



- 4) On considère maintenant un graphe pondéré où tous les poids ont une valeur négative. En considérant l'opposé des poids, peut-on résoudre le problème du plus court chemin à l'aide de l'algorithme de Dijkstra ?

Solution 4. En considérant l'opposé des poids on obtient certes un graphe pondéré positivement mais le problème de chemin de poids minimal se transforme en un problème équivalent de chemin de poids maximal. Dans ce cas, Dijkstra ne peut rien pour vous.

- 5) Pour un graphe quelconque, comment modifier l'algorithme de Dijkstra de façon à ce qu'il trouve le chemin le plus court du sommet initial a à un sommet t supposé donné ?

Solution 5. À la base du Lemme donné dans la justification de l'algorithme de Dijkstra on ajoute le test :

```
if (t=u) then do {print dist*(u); STOP}
```

- 6) Pour un graphe quelconque, comment modifier l'algorithme considéré dans la première question pour faire la même chose ?

Solution 6. On ne peut pas accélérer l'algorithme de Bellman-Ford dans le cas où le sommet initial et le sommet final sont connus en avance.

— Rappel : présence de valeurs négatives sur les arcs

Algorithm 2 Bellman-Ford (G, l, s).

Require: Graphe $G=(V, E)$ directed, edge weights $\{l_e : e \in E\}$ with no negative cycles; vertex $s \in V$

Ensure: For all vertices $u \in V$ reachable from s , $dist(u)$ is set to the shortest distance from s to u ;

```

1: for all  $u \in V$  do
2:    $dist(u) \leftarrow \infty$ 
3:    $prev(u) \leftarrow nil$ 
4: end for
5:  $dist(s) \leftarrow 0$ 
6:  $k \leftarrow 1$ 
7: while  $(k \leq |V| - 1)$  do
8:   for all edges  $(u, v) \in E$  do
9:      $dist(v) \leftarrow \min\{dist(v), dist(u) + l(u, v)\}$ 
10:  end for
11:   $k \leftarrow k + 1$ 
12: end while

```

Algorithm 3 Bellman-Ford_bis (G, l, s).

Require: Graphe $G=(V, E)$ directed, edge weights $\{l_e : e \in E\}$; vertex $s \in V$

Ensure: $dist^k(u)$ is set to the shortest distance from s to u over all paths with at most k arcs. It also detects the presence of negative cycle.

```

1: Initialisation :  $\forall u \in V : dist^0(u) \leftarrow \infty$ ;  $dist^0(s) \leftarrow 0$ ;  $stable \leftarrow false$ ;  $k \leftarrow 1$ ;
2: while  $((k \leq |V|) \text{ and } (nonstable))$  do
3:    $dist^k(s) \leftarrow 0$ 
4:   for all vertices  $v \in V \setminus \{s\}$  do
5:      $dist^k(v) = \min\{dist^{k-1}(v), \min_{u \in \Gamma^{-1}(v)} dist^{k-1}(u) + l(u, v)\}$ 
6:   end for
7:    $stable \leftarrow (dist^k(v) = dist^{k-1}(v), \text{ for all } v \in V)$ ;  $k \leftarrow k + 1$ ;
8: end while
9: if  $(k = |V| + 1)$  then  $\exists$  negative cycle
10: end if

```
